

Checked vs. Unchecked Exceptions und Errors in Java

Benjamin Malicke ©

10. März 2026

1 Unterschied zwischen checked und unchecked Exceptions

In Java gibt es zwei Hauptarten von Exceptions: *checked* und *unchecked* Exceptions.

1.1 Checked Exceptions

- Werden **zur Compile-Zeit** vom Compiler überprüft.
- Methoden, die eine checked Exception auslösen können, müssen sie entweder mit **throws** in der Methodensignatur deklarieren oder sie im Methodenrumpf mit **try-catch** behandeln.
- Typische Beispiele: `IOException`, `SQLException`.
- Ziel: Den Entwickler dazu zwingen, mit vorhersehbaren Fehlerfällen (z.B. Datei nicht gefunden, Datenbank nicht erreichbar) explizit umzugehen.

1.2 Unchecked Exceptions

- Werden **nicht vom Compiler erzwungen** – sie können, müssen aber nicht explizit behandelt oder deklariert werden.
- Technisch sind es alle Exceptions, die von `RuntimeException` erben.
- Treten oft bei Programmierfehlern oder logischen Fehlern auf, z.B. `NullPointerException`, `ArrayIndexOutOfBoundsException`.
- Ziel: Keine übermäßige “Checked-Exception-Bürokratie” für Programmierfehler, die man eher durch sauberes Design vermeiden sollte.

1.3 Gegenüberstellung

	Checked Exceptions	Unchecked Exceptions
Überprüfung	Zur Compile-Zeit	Zur Laufzeit
Behandlung	try-catch oder throws zwingend	Behandlung optional
Typische Ursache	Erwartbare, externe Fehler (I/O, DB, Netzwerk)	Programmierfehler, Logikfehler
Basisklasse	<code>Exception</code> (ohne <code>RuntimeException</code>)	<code>RuntimeException</code>
Beispiele	<code>IOException</code> , <code>ParseException</code>	<code>NullPointerException</code> , <code>IllegalArgumentException</code>

2 Errors in Java

Neben Exceptions gibt es in Java auch **Errors**. Sie sind Klassen, die von `java.lang.Error` erben.

- **Errors** signalisieren meist schwerwiegende Probleme der Laufzeitumgebung (z.B. kein Speicher mehr).
- Sie sollten im Normalfall **nicht** mit `try-catch` gefangen werden, da das Programm sich in einem Zustand befindet, aus dem es sich oft nicht sinnvoll erholen kann.
- Beispiele: `OutOfMemoryError`, `StackOverflowError`, `NoClassDefFoundError`.
- Im Gegensatz zu Exceptions repräsentieren Errors meist Probleme, die außerhalb der Kontrolle der Anwendung liegen oder so gravierend sind, dass ein Weiterlaufen nicht sinnvoll ist.

3 Häufige Exceptions in Java

Die folgende Tabelle listet einige der häufigsten Exceptions in Java auf, unterteilt nach checked und unchecked Exceptions.

3.1 Checked Exceptions

Exception	Typische Bedeutung / Einsatz
<code>IOException</code>	Allgemeine Ein-/Ausgabefehler, z.B. beim Lesen/Schreiben von Dateien oder Netzwerkzugriffen.
<code>FileNotFoundException</code>	Spezialfall von <code>IOException</code> : Angegebene Datei oder Ressource existiert nicht.
<code>SQLException</code>	Fehler bei Datenbankzugriffen über JDBC (z.B. Syntaxfehler im SQL, Verbindungsprobleme).
<code>ParseException</code>	Fehler beim Parsen von Daten (z.B. Datums- oder Zahlenformate).
<code>ClassNotFoundException</code>	Eine Klasse, die dynamisch geladen werden soll, kann nicht gefunden werden.
<code>InterruptedException</code>	Ein Thread wurde unterbrochen, während er wartet oder schläft.
<code>CloneNotSupportedException</code>	<code>clone()</code> wurde auf einem Objekt aufgerufen, das <code>Cloneable</code> nicht implementiert.

3.2 Unchecked Exceptions (RuntimeExceptions)

Exception	Typische Bedeutung / Einsatz
NullPointerException	Zugriff auf Methoden/Felder einer Referenz, die null ist.
ArrayIndexOutOfBoundsException	Zugriff auf ein Array mit einem Index außerhalb des gültigen Bereichs.
IndexOutOfBoundsException	Ähnlich wie oben, aber allgemeiner (z.B. bei Listen).
IllegalArgumentException	Eine Methode erhält ein ungültiges Argument (z.B. negative Größe).
NumberFormatException	Versuch, einen String in eine Zahl zu parsen, obwohl das Format ungültig ist.
ArithmeticException	Arithmetische Fehler, z.B. Division durch Null bei Ganzzahlen.
ClassCastException	Ungültiger Typumwandlungsversuch (Cast) zwischen inkompatiblen Typen.
UnsupportedOperationException	Eine angeforderte Operation wird von einer Klasse nicht unterstützt (z.B. unveränderliche Collections).



